

Universitat Politècnica de Catalunya

ESEIAAT

Computational Engineering

Generic convection-diffusion transport equation

Gerard Martínez Vilella

Contents

1 Introduction **3**

1.1 Problem specification 3

2 Methodology **4**

2.1 Equation discretisation 4

2.2 Boundary conditions and problem definition 5

2.3 Solver and Code Structure 6

2.3.1 Problem Setup and Mesh Generation 6

2.3.2 Velocity and Diffusivity Fields 6

2.3.3 Discretization and Coefficient Assembly 7

2.3.4 Boundary Condition Enforcement 7

2.3.5 Iterative Solution 7

2.3.6 Post-processing and Visualization 7

3 Code verification **8**

4 Results analysis **9**

4.1 Diagonal case 9

4.2 Smith–Hutton case 10

4.3 Mesh convergence study 12

5 Conclusions **13**

A Code **14**

A.1 Main Script 14

A.2 Function: Setup Simulation 17

A.3 Function: Generate Mesh 18

A.4 Function: Compute Velocity Field 20

A.5 Function: Evaluate Coefficients 21

A.6 Function: Apply Boundary Conditions 25

A.7 Function: Solve System 26

A.8 Function: Plot Results 29

A.9 Script: Grid Convergence Study 31

1 Introduction

The objective of this work is to present the numerical formulation and results obtained from the resolution of the convection–diffusion transport equation applied to the classical Smith–Hutton benchmark problem. This equation is fundamental in fluid mechanics, describing how a transported scalar quantity evolves under the combined effects of convection and diffusion. The Smith–Hutton configuration serves as a canonical case for evaluating the performance, accuracy, and stability of computational fluid dynamics (CFD) algorithms.

The study focuses on a steady, two–dimensional flow field characterized by a prescribed velocity distribution. Assuming steady–state conditions eliminates temporal variations and highlights spatial changes in the scalar field. The velocity profile in this configuration generates a complex flow pattern, demanding precise discretization to balance convective and diffusive fluxes effectively.

A MATLAB code was developed to solve the governing equations using an appropriate finite–volume discretization. The program incorporates boundary conditions consistent with the physical problem and employs several numerical schemes for the convection term. The resulting scalar fields are compared against analytical and reference data from literature to ensure the fidelity of the approach.

Overall, this report demonstrates the capability of numerical methods to reproduce the expected behaviour of the convection–diffusion equation, highlighting the influence of grid density, diffusion ratio, and discretization schemes on solution quality.

1.1 Problem specification

The theoretical foundation of this study relies on the Navier–Stokes equations, which describe mass, momentum, and energy conservation in a fluid medium. For the purposes of this problem, the relevant forms are:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{1}$$

$$\rho\frac{\partial u}{\partial t} + \rho u\frac{\partial u}{\partial x} + \rho v\frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) \tag{2}$$

$$\rho\frac{\partial v}{\partial t} + \rho u\frac{\partial v}{\partial x} + \rho v\frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \mu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) + \rho g\beta(T - T_\infty) \tag{3}$$

$$\rho\frac{\partial T}{\partial t} + \rho u\frac{\partial T}{\partial x} + \rho v\frac{\partial T}{\partial y} = \frac{\lambda}{c_p}\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) + \frac{\Phi}{c_p} \tag{4}$$

From these relations, the general steady–state convection–diffusion transport equation can be expressed as:

$$\nabla \cdot (\rho \mathbf{v} \phi) = \nabla \cdot (\Gamma \nabla \phi) + S \tag{5}$$

This scalar transport equation serves as the basis for the discretization and numerical implementation developed in this work.

2 Methodology

Accurate mesh definition is essential for properly resolving the convective and diffusive contributions of the equation. The computational domain is represented by an $N \times M$ array of control volumes, where additional boundary cells are added to handle inlet, outlet, and wall conditions. This results in a $(N+2) \times (M+2)$ computational grid.

2.1 Equation discretisation

The discretized form of the transport equation follows the finite–volume formulation:

$$a_P \phi_P = a_E \phi_E + a_W \phi_W + a_N \phi_N + a_S \phi_S + b_P \tag{6}$$

with the coefficients defined as:

$$a_E = D_E A(|Pe_E|) + \max(-F_E, 0) \tag{7}$$
$$a_W = D_W A(|Pe_W|) + \max(F_W, 0) \tag{8}$$
$$a_N = D_N A(|Pe_N|) + \max(-F_N, 0) \tag{9}$$
$$a_S = D_S A(|Pe_S|) + \max(F_S, 0) \tag{10}$$

and

$$a_P = a_E + a_W + a_N + a_S + \frac{\rho_P \Delta x \Delta y}{\Delta t}, \qquad b_P = \frac{\rho_P \Delta x \Delta y}{\Delta t} \phi_P^n + S_P \Delta x \Delta y \tag{11}$$

Five different approximations for $A(|Pe|)$ were implemented:

- **Upwind Difference Scheme (UDS)** – a first–order method prioritizing stability

by selecting the upstream value according to the direction of flow.

- **Exponential Scheme (EDS)** – employs the analytical one–dimensional solution of the steady convection–diffusion equation, providing second–order accuracy.
- **Central Difference Scheme (CDS)** – a second–order linear interpolation between adjacent nodes, accurate for low Peclet numbers.
- **Hybrid Difference Scheme (HDS)** – dynamically blends UDS and CDS based on local Peclet number to maintain accuracy and stability.
- **Power–Law Scheme (PLDS)** – a polynomial approximation of the exponential model, balancing precision and computational efficiency.

2.2 Boundary conditions and problem definition

The Smith–Hutton configuration consists of a rectangular region of height H and width $2H$, with a prescribed velocity field:

$$v_x = 2y(1 - x^2) \tag{12}$$

$$v_y = -2x(1 - y^2) \tag{13}$$

The scalar ϕ satisfies the following boundary conditions:

$$\begin{aligned}
&\phi = 1 + \tanh[\alpha(2x + 1)] && \text{for } y = 0,\ x \in (-1, 0) \quad \text{(inlet)} \\
&\frac{\partial \phi}{\partial y} = 0 && \text{for } y = 0,\ x \in (0, 1) \quad \text{(outlet)} \\
&\phi = 1 - \tanh(\alpha) && \text{elsewhere}
\end{aligned}$$

This can be seen in Figure 1, where the Smith-Hutton problem domain is presented, remarking the inlet and outlet.

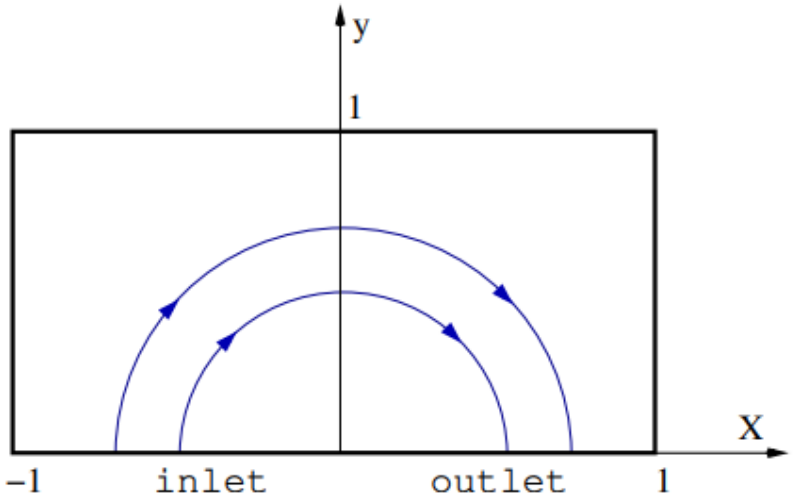


Figure 1: General schema of the proposed problem

2.3 Solver and Code Structure

The computational algorithm implements a finite volume method for solving the generic convection-diffusion transport equation. The solution proceeds through a systematic workflow combining spatial discretization, coefficient evaluation, and iterative convergence.

2.3.1 Problem Setup and Mesh Generation

The simulation begins by reading input parameters and initializing the computational domain through the `setupSimulation.m` script, which defines geometric properties (domain dimensions L and H), fluid properties (density ρ, diffusion coefficients Γ), and numerical parameters (mesh resolution $N \times M$, convergence tolerance $\epsilon = 10^{-6}$, and maximum iterations 10^6).

Following problem initialization, `generateMesh.m` constructs a uniform structured mesh with $N \times M$ cells in the x and y directions. This script computes cell centers, face centers, and geometric parameters ($s_{x,w}$, $s_{x,e}$, $s_{x,p}$ in the x-direction and $s_{y,s}$, $s_{y,n}$, $s_{y,p}$ in the y-direction) required for finite volume discretization. Boundary ghost cells are included to facilitate boundary condition implementation.

2.3.2 Velocity and Diffusivity Fields

The `computeVelocityField.m` script evaluates the velocity field at mesh nodes. Depending on the problem type, it computes either the analytical Smith-Hutton velocity field

($v_x = 2y(1 - x^2)$, $v_y = -2x(1 - y^2)$) or a uniform diagonal flow configuration.

2.3.3 Discretization and Coefficient Assembly

The `computeCoefficients.m` script implements the finite volume discretization of the transport equation using one of five discretization schemes: Upwind Differencing Scheme (UDS), Central Differencing Scheme (CDS), Hybrid Differencing Scheme (HDS), Exponential Differencing Scheme (EDS), or Power Law Differencing Scheme (PLDS). For each interior cell and the selected scheme, this script evaluates:

1. Convective fluxes at the cell faces based on velocity components,
2. Peclet numbers for adaptive scheme selection,
3. Diffusive coefficients normalized by cell geometry,
4. Neighbor coefficients (a_e, a_w, a_n, a_s) and central coefficient (a_p).

2.3.4 Boundary Condition Enforcement

The `applyBoundaryConditions.m` script modifies the coefficient matrices to enforce Dirichlet and mixed boundary conditions. For the Smith-Hutton test case, it applies analytical boundary values on left, right, and top boundaries, while the bottom boundary incorporates a hyperbolic tangent profile on the left side and a Neumann condition on the right side. For the diagonal flow case, it applies constant Dirichlet conditions on all boundaries.

2.3.5 Iterative Solution

The `solveSystem.m` script solves the resulting linear system using a nested iteration approach. An outer loop implements time-stepping for transient behavior, while an inner Gauss-Seidel loop performs spatial iterations with the stopping criterion based on the relative error between successive iterations. The solver updates phi values at all grid points (interior and boundary) until convergence is reached.

2.3.6 Post-processing and Visualization

Finally, `plotResults.m` generates visualization output, creating surface plots of the transported quantity ϕ for different diffusivity values. For the Smith-Hutton case, it also compares the numerical solution with analytical reference data through interpolation and verification plots.

The main execution sequence is coordinated by `main.m`, which orchestrates these modular components into a complete simulation framework supporting multiple discretization schemes and problem configurations.

3 Code verification

Model validation was performed by comparing outlet values with analytical solutions (Table 1) for different mesh resolutions.

Table 1: Reference values of ϕ at the outlet for different ρ/Γ ratios.

x-position	$\rho/\Gamma = 10$	$\rho/\Gamma = 10^3$	$\rho/\Gamma = 10^6$
0.0	1.989	2.0000	2.000
0.1	1.402	1.9990	2.000
0.2	1.146	1.9997	2.000
0.3	0.946	1.9850	1.999
0.4	0.775	1.8410	1.964
0.5	0.621	0.9510	1.000
0.6	0.480	0.1540	0.036
0.7	0.349	0.0010	0.001
0.8	0.227	0.0000	0.000
0.9	0.111	0.0000	0.000
1.0	0.000	0.0000	0.000

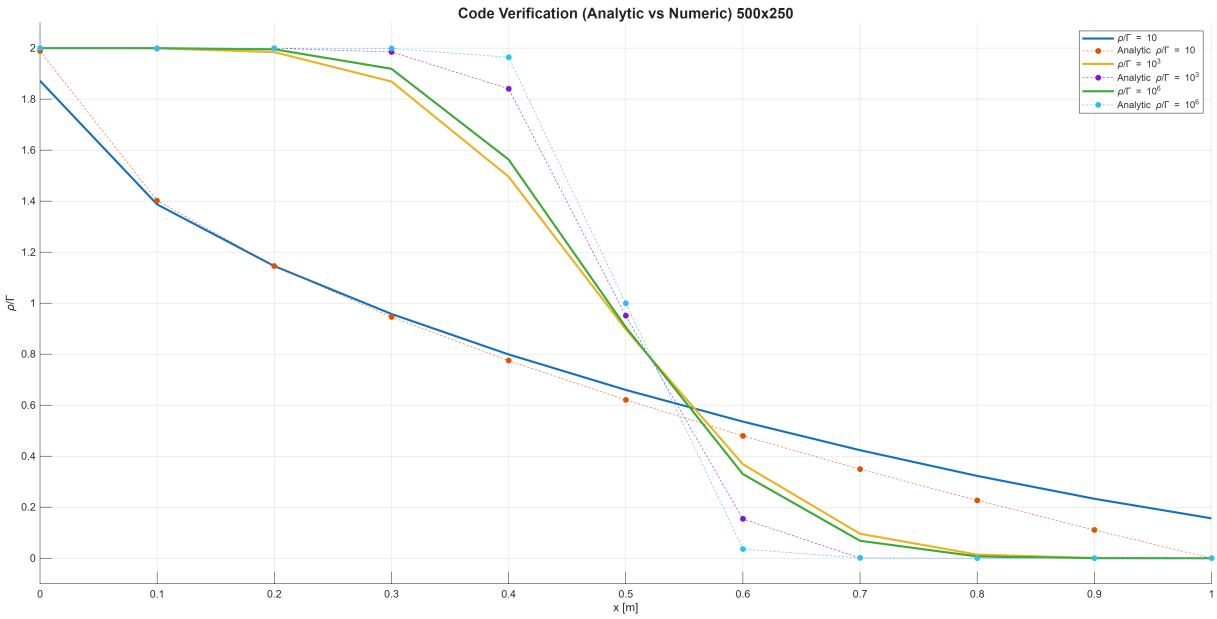


Figure 2: Verification of numerical values with analytical ones for a mesh of N = 500.

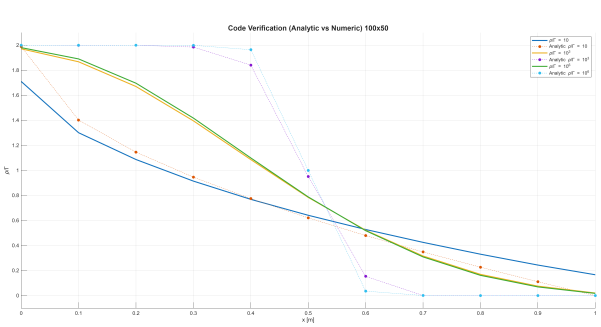


Figure 3: Verification of numerical values with analytical ones for a mesh of N = 100.

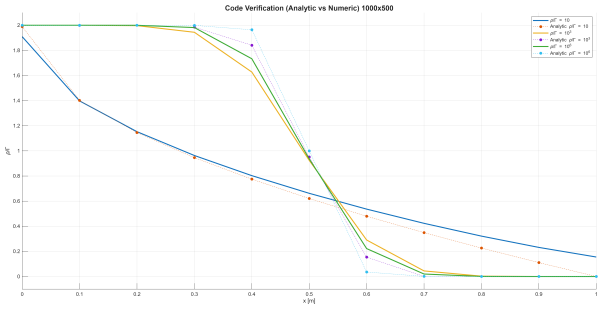


Figure 4: Verification of numerical values with analytical ones for a mesh of N = 1000.

As can be seen in Figure 2, for $N = 500$, numerical results showed close agreement with reference data, while coarser meshes ($N = 100$) (Figure 3) introduced noticeable deviations. Increasing resolution ($N = 1000$) (Figure 4) yielded more approximated curves, confirming a more detailed capture of the flux rotation field, but also increasing significantly the computational time required.. For computational resources restriction, the following studies will be done with N = 500, as verification process gave enough good results.

4 Results analysis

Two separate scenarios were examined: a diagonal flow case and the Smith–Hutton configuration. All simulations used the following baseline parameters:

- $L = 10$ m
- $N = 500$
- $\rho = 1.225$ kg/m^3
- tolerance $= 10^{-6}$
- $\Delta t = 0.1$

4.1 Diagonal case

The diagonal configuration considered $L = H = 10$ m and a 500×500 grid.

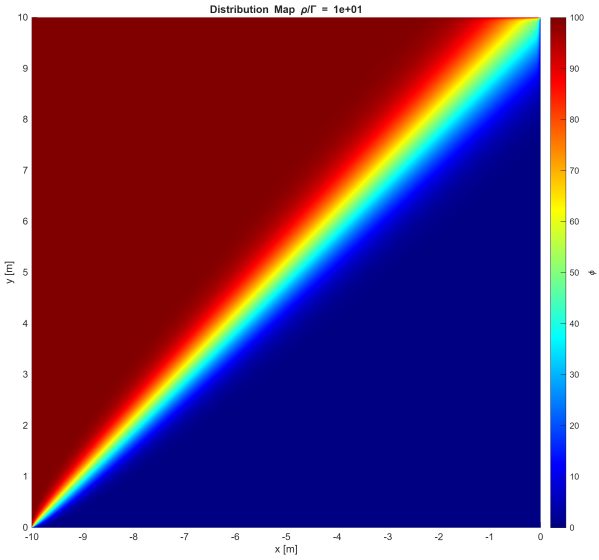


Figure 5: Diagonal case. Distribution map for $\Gamma = 10$ and $N = 500$.

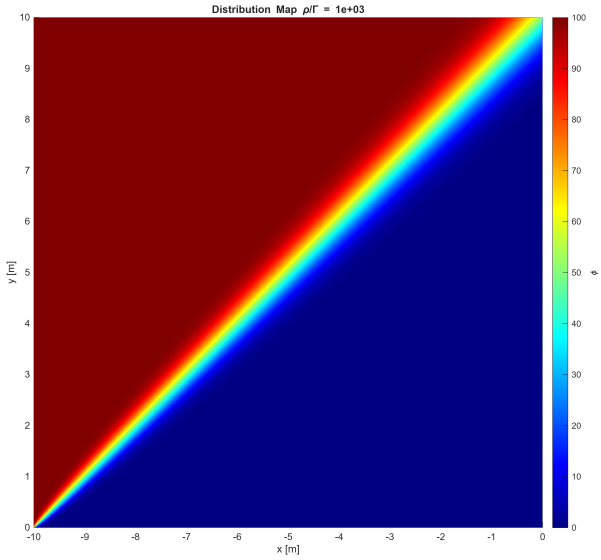


Figure 6: Diagonal case. Distribution map for $\Gamma = 10^3$ and $N = 500$.

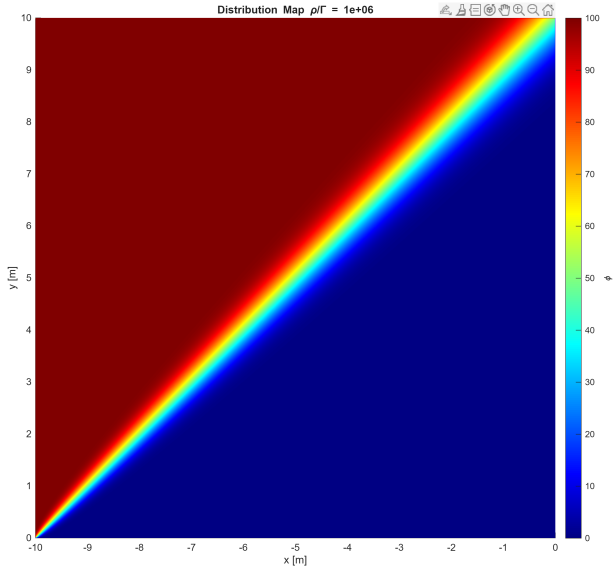


Figure 7: Diagonal case. Distribution map for $\Gamma = 10^6$ and $N = 500$.

As Γ increased from 10 to 10^6, the diffusive contribution became more significant, leading to smoother gradients and reduced dominance of convection. Lower Γ values enhanced convective effects, making the transported scalar more aligned with the velocity direction.

4.2 Smith–Hutton case

For the Smith–Hutton flow, numerical solutions were obtained for all five discretisation schemes evaluating them with the same three values of Γ as in the diagonal case.

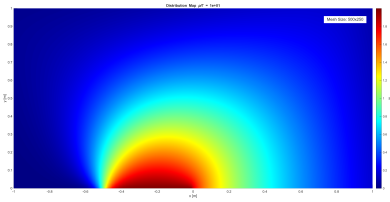


Figure 8: Smith Hutton (UDS). Distribution map for $\Gamma = 10$.

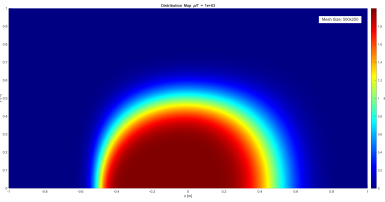


Figure 9: Smith Hutton (UDS). Distribution map for $\Gamma = 10^3$.

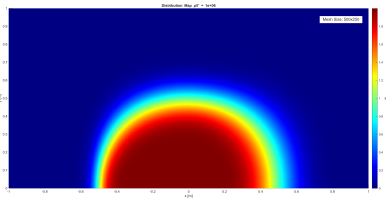


Figure 10: Smith Hutton (UDS). Distribution map for $\Gamma = 10^6$.

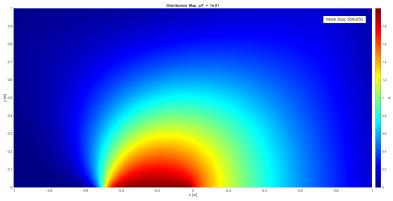


Figure 11: Smith Hutton (EDS). Distribution map for $\Gamma = 10$.

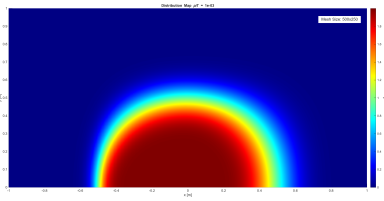


Figure 12: Smith Hutton (EDS). Distribution map for $\Gamma = 10^3$.

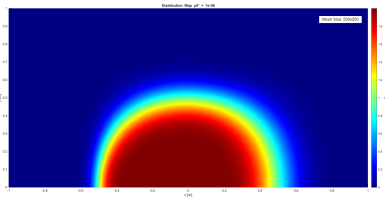


Figure 13: Smith Hutton (EDS). Distribution map for $\Gamma = 10^6$.

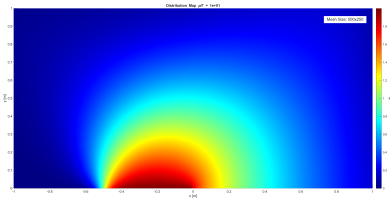


Figure 14: Smith Hutton (CDS). Distribution map for $\Gamma = 10$.

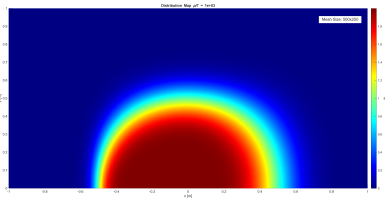


Figure 15: Smith Hutton (CDS). Distribution map for $\Gamma = 10^3$.

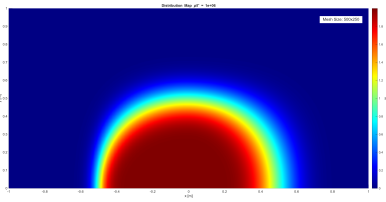


Figure 16: Smith Hutton (CDS). Distribution map for $\Gamma = 10^6$.

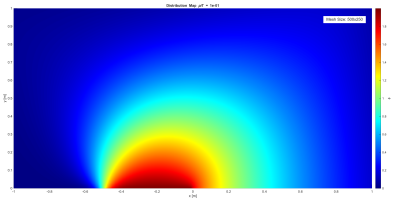


Figure 17: Smith Hutton (HDS). Distribution map for $\Gamma = 10$.

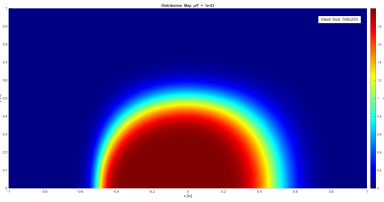


Figure 18: Smith Hutton (HDS). Distribution map for $\Gamma = 10^3$.

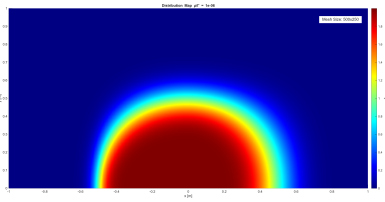


Figure 19: Smith Hutton (HDS). Distribution map for $\Gamma = 10^6$.

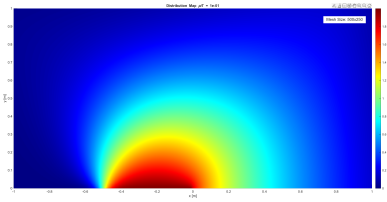


Figure 20: Smith Hutton (PLDS). Distribution map for $\Gamma = 10$.

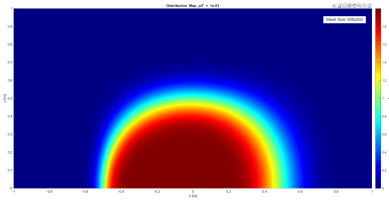


Figure 21: Smith Hutton (PLDS). Distribution map for $\Gamma = 10^3$.

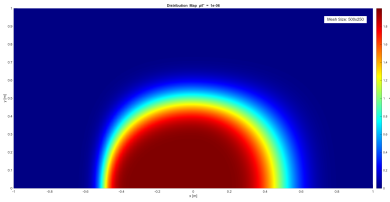


Figure 22: Smith Hutton (PLDS). Distribution map for $\Gamma = 10^6$.

The computed scalar distributions followed the expected physical behaviour. Small discrepancies in the exponential scheme for low Γ suggested minor coefficient inconsistencies, but overall trends matched the reference results. Increasing ρ/Γ emphasized convective transport, while lower ratios enhanced diffusion, producing smoother contours.

4.3 Mesh convergence study

To assess grid independence, the Hybrid Differencing Scheme (HDS) was applied on progressively refined meshes. The mean absolute error with respect to reference data decreased monotonically as the grid resolution increased, confirming that the solver is numerically convergent.

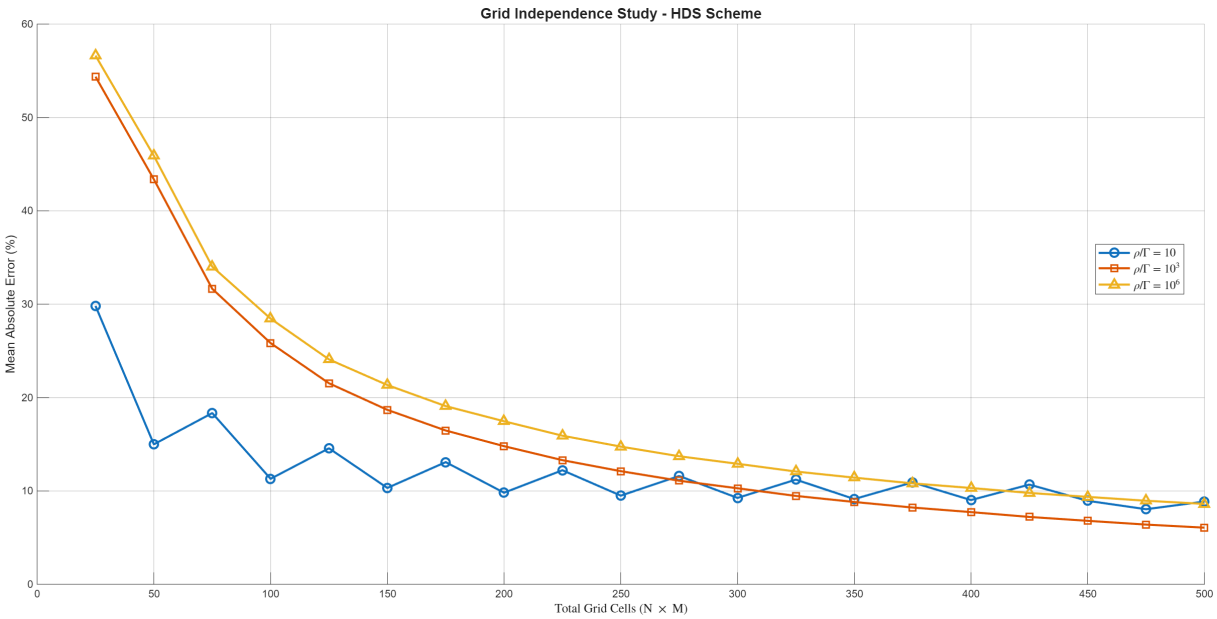


Figure 23: Mean absolute error (%) versus mesh size for different ρ/Γ values.

5 Conclusions

The developed numerical solver successfully addressed the steady–state convection–diffusion equation for both the diagonal and Smith–Hutton cases. The results confirmed the correctness of the discretisation, the reliability of the solver, and its ability to reproduce benchmark data.

Compared with previous assignments, the implementation proved more straightforward thanks to reusable solver components. The validation tests confirmed the code's precision and robustness in handling coupled convective–diffusive effects.

In conclusion, the project achieved all set objectives, delivering consistent and accurate results that validate the finite–volume implementation of the convection–diffusion transport model.

A Code

A.1 Main Script

Listing 1: Main script for convection-diffusion solver

```
%% CFD Simulation - Generic Convection-Diffusion Transport Equation
% E2 Implementation
% Assignment 2: Solving the generic convection-diffusion equation with various schemes
clearvars; close all;

%% CONFIGURATION
% Problem type:
% diag = false: Smith-Hutton test case
% diag = true: Diagonal flow test case
diag = false;

% Override for diagnostic mode
if diag
    case_study = "UDS";  % Use Upwind for diagonal case
else
    % Case study selection:
    % UDS (Upwind Differencing Scheme)
    % CDS (Central Differencing Scheme)
    % HDS (Hybrid Differencing Scheme)
    % EDS (Exponential Differencing Scheme)
    % PLDS (Power Law Differencing Scheme)
    case_study = "HDS";
end


%% READ INPUT DATA
fprintf('=== CFD Convection-Diffusion Solver ===\n');
fprintf('Case study: %s\n', case_study);
fprintf('Problem type: %s\n\n', iif(diag, 'Diagonal flow', 'Smith-Hutton'));

[Geometry, Fluid, Numerical, BoundaryConditions] = setupSimulation(diag);

%% GENERATE MESH
fprintf('Generating mesh...\n');
[nod_x, nod_y, center_x, center_y, sx_w, sx_e, sx_p, sy_s, sy_n, sy_p] = generateMesh(Geometry, Numerical);

% Cell dimensions
```

```
A_x = Geometry.L / Numerical.N;
A_y = Geometry.H / Numerical.M;

% Create meshgrid for visualization and velocity computation
x = nod_x - Geometry.H;
[surf_x, surf_y] = meshgrid(x, fliplr(nod_y));
surf_x = surf_x';
surf_y = fliplr(surf_y');

fprintf('Mesh: %d x %d cells\n', Numerical.N, Numerical.M);
fprintf('Cell size: dx = %.4f m, dy = %.4f m\n\n', A_x, A_y);

%% COMPUTE VELOCITY FIELD
fprintf('Computing velocity field...\n');
[v_x, v_y] = computeVelocityField(surf_x, surf_y, Fluid, diag);

%% SOLVER LOOP FOR DIFFERENT GAMMA VALUES
phi_def = zeros(Numerical.N+2, Numerical.M+2, length(Fluid.gamma));

for l = 1:length(Fluid.gamma)
    fprintf('--- Solving for rho/Gamma = %.0e ---\n', Fluid.GAMMA(l));

    % Initialize phi field
    phi_in = ones(Numerical.N+2, Numerical.M+2) * BoundaryConditions.
       phi_inlet;
    phi_prev = phi_in;

    % Evaluate coefficients
    fprintf('Evaluating coefficients...\n');
    [a_n, a_s, a_e, a_w, a_p, a_p_base, b_p] = computeCoefficients(v_x,
       v_y, Numerical, Fluid, ...
                                                                  sx_w,
                                                                     sx_e
                                                                     ,
                                                                     sx_p
                                                                     ,
                                                                     sy_s
                                                                     ,
                                                                     sy_n
                                                                     ,
                                                                     sy_p
                                                                     ,
                                                                     ...
                                                                  Fluid.
                                                                     gamma
                                                                     (l)
                                                                     ,
```

```
                                                                    case_study
                                                                    ,
                                                                    A_x
                                                                    ,
                                                                    A_y
                                                                    );

    % Apply boundary conditions
    fprintf('Applying boundary conditions...\n');
    [a_n, a_s, a_e, a_w, a_p, b_p] = applyBoundaryConditions(a_n, a_s,
        a_e, a_w, a_p, a_p_base, b_p, ...
                                                          surf_x,
                                                             Numerical
                                                             , diag,

                                                             BoundaryConditions
                                                             );

    % Solve system
    fprintf('Solving system...\n');
    [phi, iter_outer, iter_inner] = solveSystem(a_n, a_s, a_e, a_w, a_p,
         a_p_base, b_p, ...
                                                 phi_in, phi_prev,
                                                    Numerical);

    fprintf('Converged: Outer iterations = %d, Total inner iterations =
        %d\n\n', iter_outer, iter_inner);

    % Store solution
    phi_def(:, :, l) = phi;
end

%% PLOT RESULTS
fprintf('Generating plots...\n');
plotResults(surf_x, surf_y, phi_def, nod_x, nod_y, Geometry, Fluid, diag
    , case_study);

fprintf('\n=== Simulation completed successfully! ===\n');

%% HELPER FUNCTION
function out = iif(condition, true_val, false_val)
    % Inline if-else function
    if condition
        out = true_val;
    else
        out = false_val;
    end
```

```
end
```

A.2 Function: Setup Simulation

Listing 2: Function: Setup Simulation

```
function [Geometry, Fluid, Numerical, BoundaryConditions] =
    setupSimulation(diag)
% setupSimulation Reads and sets up all input parameters for convection-
    diffusion simulation
%
% This function defines all the physical, geometrical, and numerical
    parameters
% required for solving the generic convection-diffusion transport
    equation.
%
% INPUTS:
%   diag        - Diagonal flow test case (1 = diagonal, 0 = Smith-Hutton
    )
%
% OUTPUTS:
%   Geometry            - Structure with L, H, x_in, y_in, x_out, y_out
%   Fluid               - Structure with rho, GAMMA, gamma
%   Numerical           - Structure with N, M, tolerance, maxIt,
    time_step
%   BoundaryConditions - Structure with phi_inlet
%
    ------------------------------------------------------------------------


    % Geometry properties
    Geometry.L = 10;  % [m] Domain length

    if diag
        Geometry.H = Geometry.L;  % Square domain for diagonal case
    else
        Geometry.H = Geometry.L / 2;  % Smith-Hutton case
    end

    Geometry.x_in = -Geometry.L / 2;
    Geometry.y_in = Geometry.H;
    Geometry.x_out = Geometry.L / 2;
    Geometry.y_out = 0;

    % Fluid properties
    Fluid.rho = 1.225;  % [kg/m^3] Density
```

```
    Fluid.GAMMA = [10, 10^3, 10^6];  % Diffusion coefficient values
    Fluid.gamma = Fluid.rho ./ Fluid.GAMMA;  % Normalized diffusion

    % Diagonal case specific velocity
    if diag
        Fluid.v_0 = 10;  % [m/s] Velocity magnitude for diagonal flow
    end

    % Numerical parameters
    Numerical.N = 500;  % Number of cells in x-direction
    Numerical.M = round(Geometry.H / Geometry.L * Numerical.N);  % Cells
        in y-direction

    if diag
        Numerical.M = Numerical.N;  % Square mesh for diagonal case
    end

    Numerical.tolerance = 1e-6;  % Convergence tolerance
    Numerical.maxIt = 1e6;  % Maximum iterations
    Numerical.time_step = 0.1;  % Time step for transient solver

    % Boundary conditions
    BoundaryConditions.phi_inlet = 1;  % Generic phi inlet value

    % Diagonal case boundary values
    if diag
        BoundaryConditions.phi_left = 100;
        BoundaryConditions.phi_right = 0;
        BoundaryConditions.phi_bottom = 0;
        BoundaryConditions.phi_top = 100;
    end

end
```

A.3 Function: Generate Mesh

Listing 3: Function: Generate Mesh

```
function [nod_x, nod_y, center_x, center_y, sx_w, sx_e, sx_p, sy_s, sy_n
    , sy_p] = generateMesh(Geometry, Numerical)
% GENERATEMESH Creates a structured mesh for convection-diffusion
    problem
%
% This function generates a uniform computational grid with cell centers
    ,
```

```
% face positions, and geometric parameters needed for finite volume discretization.
%
% INPUTS:
%   Geometry  - Structure containing L, H (domain dimensions)
%   Numerical - Structure containing N, M (number of cells)
%
% OUTPUTS:
%   nod_x, nod_y       - Node coordinates (cell centers) including boundaries
%   center_x, center_y - Cell face centers
%   sx_w, sx_e, sx_p   - Geometric parameters in x-direction (west, east, P)
%   sy_s, sy_n, sy_p   - Geometric parameters in y-direction (south, north, P)
%
   ------------------------------------------------------------------------


    L = Geometry.L;
    H = Geometry.H;
    N = Numerical.N;
    M = Numerical.M;

    % Cell dimensions
    A_x = L / N;
    A_y = H / M;

    % Generate cell face centers (0:N and 0:M)
    center_x = (0:N) * A_x;
    center_y = (0:M) * A_y;

    % Generate cell centers (interior nodes)
    nodvec_x = ((0:N-1) * A_x) + (A_x / 2);
    nodvec_y = ((0:M-1) * A_y) + (A_y / 2);

    % Add boundary nodes (ghost cells)
    nod_x = [0, nodvec_x, L];
    nod_y = [0, nodvec_y, H];

    % Preallocate geometric parameter arrays
    sx_w = zeros(size(nod_x));
    sx_e = zeros(size(nod_x));
    sx_p = zeros(size(nod_x));
    sy_n = zeros(size(nod_y));
    sy_s = zeros(size(nod_y));
    sy_p = zeros(size(nod_y));
```

```
46
47      % X-direction geometric parameters (vectorized)
48      % Interior cells
49      sx_w(2:end-1) = nod_x(2:end-1) - nod_x(1:end-2);
50      sx_e(2:end-1) = nod_x(3:end) - nod_x(2:end-1);
51
52      % Ensure center_x matches nod_x size
53      if numel(center_x) ~= numel(nod_x)
54          center_x = linspace(min(nod_x), max(nod_x), numel(nod_x));
55      end
56      sx_p(2:end-1) = center_x(2:end-1) - center_x(1:end-2);
57
58      % X-direction boundary conditions
59      sx_w(1) = 0;
60      sx_w(end) = nod_x(end) - nod_x(end-1);
61      sx_e(1) = nod_x(2) - nod_x(1);
62      sx_e(end) = 0;
63      sx_p(1) = center_x(2) / 2;
64      sx_p(end) = (center_x(end) - center_x(end-1)) / 2;
65
66      % Y-direction geometric parameters (vectorized)
67      % Interior cells
68      sy_s(2:end-1) = nod_y(2:end-1) - nod_y(1:end-2);
69      sy_n(2:end-1) = nod_y(3:end) - nod_y(2:end-1);
70
71      % Ensure center_y matches nod_y size
72      if numel(center_y) ~= numel(nod_y)
73          center_y = linspace(min(nod_y), max(nod_y), numel(nod_y));
74      end
75      sy_p(2:end-1) = center_y(2:end-1) - center_y(1:end-2);
76
77      % Y-direction boundary conditions
78      sy_s(1) = 0;
79      sy_s(end) = nod_y(end) - nod_y(end-1);
80      sy_n(1) = nod_y(2) - nod_y(1);
81      sy_n(end) = 0;
82      sy_p(1) = center_y(2) / 2;
83      sy_p(end) = (center_y(end) - center_y(end-1)) / 2;
84
85  end
```

A.4 Function: Compute Velocity Field

Listing 4: Function: Compute Velocity Field

```
1  function [v_x, v_y] = computeVelocityField(surf_x, surf_y, Fluid, diag)
```

```
% COMPUTEVELOCITYFIELD Computes the velocity field for the convection-diffusion problem
%
% This function calculates the velocity components based on the problem type:
% - Smith-Hutton test case: v_x = 2*y*(1-x^2), v_y = -2*x*(1-y^2)
% - Diagonal flow: v_x = v_0*cos(45  ), v_y = v_0*sin(45  )
%
% INPUTS:
%   surf_x, surf_y - Meshgrid coordinates
%   Fluid          - Structure containing fluid properties (v_0 for diagonal case)
%   diag           - Flag: 1 for diagonal flow, 0 for Smith-Hutton
%
% OUTPUTS:
%   v_x - Velocity component in x-direction
%   v_y - Velocity component in y-direction
%
    ------------------------------------------------------------------------



    if diag
        % Diagonal flow case: uniform velocity at 45 degrees
        v_x = Fluid.v_0 * cosd(45) * ones(size(surf_x));
        v_y = Fluid.v_0 * sind(45) * ones(size(surf_y));
    else
        % Smith-Hutton test case: analytical velocity field
        v_x = 2 * surf_y .* (1 - surf_x.^2);
        v_y = -2 * surf_x .* (1 - surf_y.^2);
    end

end
```

A.5 Function: Evaluate Coefficients

Listing 5: Function: Evaluate Coefficients

```
function [a_n, a_s, a_e, a_w, a_p, a_p_base, b_p] = computeCoefficients(
    v_x, v_y, Numerical, Fluid, ...
                                                                    sx_w
                                                                        ,
                                                                    sx_e
                                                                        ,
                                                                    sx_p
```

```
                                                                    ,
                                                                    sy_s
                                                                    ,
                                                                    sy_n
                                                                    ,
                                                                    sy_p
                                                                    ,
                                                                    ...
3                                                                   gamma_val
                                                                    ,
                                                                    case_study
                                                                    ,
                                                                    A_x
                                                                    ,
                                                                    A_y
                                                                    )

4   % COMPUTECOEFFICIENTS Evaluates discretization coefficients for
        convection-diffusion equation
5   %
6   % This function computes the coefficients for the finite volume
        discretization
7   % of the generic convection-diffusion transport equation using various
        schemes.
8   %
9   % INPUTS:
10  %   v_x, v_y    - Velocity field components
11  %   Numerical   - Structure with N, M, time_step
12  %   Fluid       - Structure with rho
13  %   sx_w, sx_e, sx_p - Geometric parameters in x-direction
14  %   sy_s, sy_n, sy_p - Geometric parameters in y-direction
15  %   gamma_val   - Current diffusion coefficient value
16  %   case_study - Discretization scheme (1=Upwind, 2=Exp, 3=CDS, 4=HDS,
        5=PLDS)
17  %   A_x, A_y    - Cell face areas
18  %
19  % OUTPUTS:
20  %   a_n, a_s, a_e, a_w - Neighbor coefficients (north, south, east, west
        )
```

```
%   a_p                 - Central coefficient
%   a_p_base            - Transient term coefficient
%   b_p                 - Source term
%
    ------------------------------------------------------------------------


    N = Numerical.N;
    M = Numerical.M;
    rho = Fluid.rho;
    time_step = Numerical.time_step;

    % Initialize coefficient matrices (N+2, M+2)
    a_n = zeros(N+2, M+2);
    a_s = zeros(N+2, M+2);
    a_e = zeros(N+2, M+2);
    a_w = zeros(N+2, M+2);
    a_p = ones(N+2, M+2);
    a_p_base = zeros(N+2, M+2);
    b_p = zeros(N+2, M+2);

    % Compute diffusion coefficients (vectorized where possible)
    D_n = (gamma_val * A_x) ./ sy_n(2:M+1);
    D_s = (gamma_val * A_x) ./ sy_s(2:M+1);
    D_e = (gamma_val * A_y) ./ sx_e(2:N+1);
    D_w = (gamma_val * A_y) ./ sx_w(2:N+1);

    % Loop over interior cells
    for j = 2:M+1
        for i = 2:N+1
            % Compute convective fluxes at cell faces
            F_n = rho * (v_y(i, j+1) + v_y(i, j)) * A_x / 2;
            F_s = rho * (v_y(i, j-1) + v_y(i, j)) * A_x / 2;
            F_e = rho * (v_x(i+1, j) + v_x(i, j)) * A_y / 2;
            F_w = rho * (v_x(i-1, j) + v_x(i, j)) * A_y / 2;

            % Compute Peclet numbers
            p_n = F_n / D_n(j-1);
            p_s = F_s / D_s(j-1);
            p_e = F_e / D_e(i-1);
            p_w = F_w / D_w(i-1);

            % Select discretization scheme
            switch case_study
                case "UDS"
                    A_n = 1;
                    A_s = 1;
```

```
                    A_e = 1;
                    A_w = 1;

                case "CDS"
                    A_n = abs((abs(p_n)/2) - 1);
                    A_s = abs((abs(p_s)/2) - 1);
                    A_e = abs((abs(p_e)/2) - 1);
                    A_w = abs((abs(p_w)/2) - 1);

                case "HDS"
                    A_n = max(0, 1 - (abs(p_n)/2));
                    A_s = max(0, 1 - (abs(p_s)/2));
                    A_e = max(0, 1 - (abs(p_e)/2));
                    A_w = max(0, 1 - (abs(p_w)/2));

                case "EDS"
                    A_n = abs(p_n) / (exp(abs(p_n)) - 1);
                    A_s = abs(p_s) / (exp(abs(p_s)) - 1);
                    A_e = abs(p_e) / (exp(abs(p_e)) - 1);
                    A_w = abs(p_w) / (exp(abs(p_w)) - 1);

                case "PLDS"
                    A_n = max(0, 1 - (abs(p_n)/2)^5);
                    A_s = max(0, 1 - (abs(p_s)/2)^5);
                    A_e = max(0, 1 - (abs(p_e)/2)^5);
                    A_w = max(0, 1 - (abs(p_w)/2)^5);

                otherwise
                    error('The case_study selected is not available');
            end

            % Compute coefficients using the selected scheme
            a_n(i, j) = D_n(j-1) * A_n + max(-F_n, 0);
            a_s(i, j) = D_s(j-1) * A_s + max(F_s, 0);
            a_e(i, j) = D_e(i-1) * A_e + max(-F_e, 0);
            a_w(i, j) = D_w(i-1) * A_w + max(F_w, 0);

            % Transient term
            a_p_base(i, j) = rho * sx_p(i) * sy_p(j) / time_step;

            % Central coefficient
            a_p(i, j) = a_e(i, j) + a_w(i, j) + a_n(i, j) + a_s(i, j) +
                a_p_base(i, j);
        end
    end

end
```

A.6 Function: Apply Boundary Conditions

Listing 6: Function: Apply Boundary Conditions

```
function [a_n, a_s, a_e, a_w, a_p, b_p] = applyBoundaryConditions(a_n,
    a_s, a_e, a_w, a_p, a_p_base, b_p, ...
                                                                surf_x
                                                                    ,
                                                                    Numerical
                                                                    ,
                                                                    diag
                                                                    ,
                                                                    BoundaryConditi
                                                                    )
% APPLYBOUNDARYCONDITIONS Applies boundary conditions to coefficient
    matrices
%
% This function modifies the coefficient matrices to enforce boundary
    conditions
% for either the Smith-Hutton test case or the diagonal flow case.
%
% INPUTS:
%   a_n, a_s, a_e, a_w - Neighbor coefficients
%   a_p                - Central coefficient
%   a_p_base           - Transient term coefficient
%   b_p                - Source term
%   surf_x             - X-coordinates of mesh grid
%   Numerical          - Structure with N, M
%   diag               - Flag: 1 for diagonal, 0 for Smith-Hutton
%   BoundaryConditions - Structure with boundary values
%
% OUTPUTS:
%   a_n, a_s, a_e, a_w - Modified neighbor coefficients
%   a_p                - Modified central coefficient
%   b_p                - Modified source term
%
    ------------------------------------------------------------------------


    N = Numerical.N;
    M = Numerical.M;

    if diag
        % Diagonal flow case: simple Dirichlet boundaries
        b_p(1, 1:M+2) = BoundaryConditions.phi_left;      % Left wall
        b_p(N+2, 1:M+2) = BoundaryConditions.phi_right;   % Right wall
        b_p(1:N+2, 1) = BoundaryConditions.phi_bottom;    % Bottom wall
        b_p(1:N+2, M+2) = BoundaryConditions.phi_top;     % Top wall
```

```
33
34      else
35          % Smith-Hutton test case boundaries
36          % Left, right, and top boundaries
37          b_p(1, 2:M+1) = 1 - tanh(10);
38          b_p(N+2, 2:M+1) = 1 - tanh(10);
39          b_p(2:N+1, M+2) = 1 - tanh(10);
40
41          % Bottom boundary (varies with x position)
42          for i = 2:N+1
43              if surf_x(i, M+2) < 0
44                  % Left side of bottom boundary
45                  b_p(i, 1) = 1 + tanh(10 * (2 * surf_x(i, M+2) + 1));
46              else
47                  % Right side of bottom boundary (Neumann condition)
48                  b_p(i, 1) = a_n(i, 1) + a_s(i, 1) + a_e(i, 1) + a_w(i,
                        1) + a_p_base(i, 1);
49                  a_n(i, 1) = 1;
50              end
51          end
52      end
53
54  end
```

A.7 Function: Solve System

Listing 7: Function: Solve System

```
1   function [phi, iter_outer, iter_inner] = solveSystem(a_n, a_s, a_e, a_w,
        a_p, a_p_base, b_p, ...
2                                                       phi_in, phi_prev,
                                                           Numerical)
3   % SOLVESYSTEM Solves the convection-diffusion equation using nested
        iteration
4   %
5   % This function implements a two-level iterative solver:
6   % - Outer loop: Time-stepping for transient behavior
7   % - Inner loop: Gauss-Seidel iteration for spatial discretization
8   %
9   % INPUTS:
10  %   a_n, a_s, a_e, a_w - Neighbor coefficients
11  %   a_p                - Central coefficient
12  %   a_p_base           - Transient term coefficient
13  %   b_p                - Source term (boundary conditions)
14  %   phi_in             - Initial guess for phi
15  %   phi_prev           - Previous time step solution
```

```
%   Numerical           - Structure with N, M, tolerance, maxIt,
    time_step
%
% OUTPUTS:
%   phi         - Converged solution
%   iter_outer - Number of outer (time) iterations
%   iter_inner - Total number of inner (Gauss-Seidel) iterations
%
    ---------------------------------------------------------------------------


    N = Numerical.N;
    M = Numerical.M;
    tolerance = Numerical.tolerance;
    maxIt = Numerical.maxIt;
    time_step = Numerical.time_step;

    phi = phi_in;
    iter_inner = 0;

    % Outer loop: Time-stepping
    for iter_outer = 1:maxIt
        % Update source term with transient contribution
        b_p(2:N+1, 2:M+1) = a_p_base(2:N+1, 2:M+1) .* phi_prev(2:N+1, 2:
            M+1);

        % Inner loop: Gauss-Seidel iteration
        for m = 1:maxIt
            phi_old = phi;

            % Gauss-Seidel update (vectorized for interior nodes)
            for j = 2:M+1
                for i = 2:N+1
                    phi(i, j) = (a_e(i, j) * phi(i+1, j) + ...
                                 a_w(i, j) * phi(i-1, j) + ...
                                 a_n(i, j) * phi(i, j+1) + ...
                                 a_s(i, j) * phi(i, j-1) + ...
                                 b_p(i, j)) / a_p(i, j);
                end
            end

            % Boundary updates (vectorized)
            % Top boundary (j = M+2)
            i = 2:N+1;
            phi(i, M+2) = (a_e(i, M+2) .* phi(i+1, M+2) + ...
                           a_w(i, M+2) .* phi(i-1, M+2) + ...
                           a_s(i, M+2) .* phi(i, M+1) + ...
```

```
                            b_p(i, M+2)) ./ a_p(i, M+2);

            % Bottom boundary (j = 1)
            phi(i, 1) = (a_e(i, 1) .* phi(i+1, 1) + ...
                         a_w(i, 1) .* phi(i-1, 1) + ...
                         a_n(i, 1) .* phi(i, 2) + ...
                         b_p(i, 1)) ./ a_p(i, 1);

            % Left boundary (i = 1)
            j = 2:M+1;
            phi(1, j) = (a_e(1, j) .* phi(2, j) + ...
                         a_n(1, j) .* phi(1, j+1) + ...
                         a_s(1, j) .* phi(1, j-1) + ...
                         b_p(1, j)) ./ a_p(1, j);

            % Right boundary (i = N+2)
            phi(N+2, j) = (a_w(N+2, j) .* phi(N+1, j) + ...
                           a_n(N+2, j) .* phi(N+2, j+1) + ...
                           a_s(N+2, j) .* phi(N+2, j-1) + ...
                           b_p(N+2, j)) ./ a_p(N+2, j);

            % Check inner convergence
            err_phi = max(abs(phi - phi_old), [], 'all');
            iter_inner = iter_inner + 1;

            if err_phi < tolerance
                break;
            end
        end

        % Check outer convergence (transient)
        err = abs(phi_prev - phi);
        error_value = max(err / time_step, [], 'all');
        phi_prev = phi;

        if error_value <= tolerance
            break;
        end
    end

    if iter_outer == maxIt
        warning('Maximum outer iterations reached without convergence');
    end

end
```

A.8 Function: Plot Results

Listing 8: Function: Plot Results

```
function plotResults(surf_x, surf_y, phi_def, nod_x, nod_y, Geometry, Fluid, diag, case_study)
% PLOTRESULTS Generates visualization plots for the convection-diffusion solution
%
% This function creates surface plots of the phi field for different gamma values
% and optionally compares with analytical solutions for the Smith-Hutton case.
%
% INPUTS:
%   surf_x, surf_y - Meshgrid coordinates
%   phi_def        - Solution array (N+2, M+2, num_gamma)
%   nod_x, nod_y   - Node coordinates
%   Geometry       - Structure with H
%   Fluid          - Structure with gamma
%   diag           - Bool: true for diagonal, false for Smith-Hutton
%   case_study     - case_study selected
%
    ------------------------------------------------------------------------


    gamma = Fluid.gamma;
    H = Geometry.H;
    s = size(surf_x) - 2;
    mesh_size = s(1) + "x" + s(2);

    if ~diag
        % Smith-Hutton case: Compare with analytical solution
        x = [0.000, 0.100, 0.200, 0.300, 0.400, 0.500, 0.600, 0.700, 0.800, 0.900, 1.000]';
        p = [1.989, 1.402, 1.146, 0.946, 0.775, 0.621, 0.480, 0.349, 0.227, 0.111, 0.000;
             2.00,  1.999, 1.9997, 1.985, 1.841, 0.951, 0.154, 0.001, 0.000, 0.000, 0.000;
             2.000, 2.000, 2.000, 1.999, 1.964, 1.000, 0.036, 0.001, 0.000, 0.000, 0.000]';

        phi_n = zeros(length(x), size(p, 2));

        % Interpolate numerical solution at analytical points
        for i = 1:size(p, 2)
            assert(x(1) >= (nod_x(1) - H), 'You must have min(x) <= min(xx).');
```

```
            assert(x(end) <= (nod_x(end) - H), 'You must have max(xx) <= max(x).');

            int_phi = interp1(nod_x - H, phi_def(:, 1, i), x, 'linear', 'extrap');
            phi_n(:, i) = int_phi(:);
        end

        % Verification plot
        figure('Name', 'Code Verification', 'NumberTitle', 'off');
        hold on;
        for i = 1:length(gamma)
            h1 = plot(x, phi_n(:, i), 'LineWidth', 2);
            h2 = plot(x, p(:, i), '--o', 'MarkerSize', 5);

            % Fill dots
            h1.MarkerFaceColor = h1.Color;
            h2.MarkerFaceColor = h2.Color;
        end
        grid on;
        xlabel('x [m]');
        ylabel('\rho/\Gamma');
        ylim([-0.1, 2.1]);
        name = "Code Verification (Analytic vs Numeric) " + mesh_size;
        title(name, 'FontSize', 14);
        legend('\rho/\Gamma = 10', 'Analytic \rho/\Gamma = 10', ...
               '\rho/\Gamma = 10^3', 'Analytic \rho/\Gamma = 10^3', ...
               '\rho/\Gamma = 10^6', 'Analytic \rho/\Gamma = 10^6');
        hold off;
        dir = "figures/errors/Code Verification_" + mesh_size + ".png";
        exportgraphics(gca, dir, 'Resolution', 300, 'ContentType', 'image', 'BackgroundColor', 'white');
    end

    % Distribution maps for each gamma value
    for i = 1:length(gamma)
        figure('Name', sprintf('Distribution Map - rho/Gamma = %.0e', Fluid.gamma(i)), 'NumberTitle', 'off');
        surf(surf_x, surf_y, phi_def(:, :, i), 'EdgeColor', 'none', 'FaceColor', 'interp');
        view(0, 90);
        c = colorbar;
        c.Label.String = '\phi';
        colormap jet;
        title(sprintf('Distribution Map \\rho/\\Gamma = %.0e', Fluid.gamma(i)));
        % Texto tipo nota arriba a la derecha con fondo blanco
```

```
75          if ~diag
76              annotation('textbox', [0.69, 0.79, 0.25, 0.1], ...
77                  'String', "Mesh Size: " + mesh_size, ...
78                  'FitBoxToText', 'on', ...
79                  'BackgroundColor', 'white', ...
80                  'EdgeColor', 'none', ...
81                  'HorizontalAlignment', 'center', ...
82                  'VerticalAlignment', 'top', ...
83                  'FontSize', 12);
84          end
85
86          axis equal;
87          xlabel('x [m]');
88          ylabel('y [m]');
89          grid off;
90
91          if ~diag
92              xlim([-1, 1]);
93              ylim([0, 1]);
94          else
95              xlim([-10, 0]);
96              ylim([0, 10]);
97          end
98          if diag
99              dir = "figures/UDS/diag/Distribution_map_" + Fluid.GAMMA(i)
                    + "_" + mesh_size + ".png";
100         else
101             dir = "figures/" + case_study + "/Distribution_map_" + Fluid
                    .GAMMA(i) + "_" + mesh_size + ".png";
102         end
103         % Obtener la carpeta de la ruta
104         folder = fileparts(dir);
105
106         % Crear la carpeta si no existe
107         if ~exist(folder, "dir")
108             mkdir(folder);
109         end
110         exportgraphics(gca, dir, 'Resolution', 300, 'ContentType', '
                image', 'BackgroundColor', 'white');
111     end
112
113 end
```

A.9 Script: Grid Convergence Study

Listing 9: Script: Grid Convergence Study

```
%% Grid Independence Study
% Assesses convergence of the HDS scheme on progressively refined meshes
% Computes mean absolute error against analytical Smith-Hutton reference data

clearvars; close all;

fprintf('=== Grid Independence Study (HDS) ===\n\n');

mesh_resolutions = linspace(25, 500, 20);
mae_values = zeros(length(mesh_resolutions), 3);
num_cells = zeros(length(mesh_resolutions), 1);

analytical_data_x = [0.000, 0.100, 0.200, 0.300, 0.400, 0.500, 0.600, 0.700, 0.800, 0.900, 1.000]';
analytical_data_p = [1.989, 1.402, 1.146, 0.946, 0.775, 0.621, 0.480, 0.349, 0.227, 0.111, 0.000;
                     2.00,  1.999, 1.9997, 1.985, 1.841, 0.951, 0.154, 0.001, 0.000, 0.000, 0.000;
                     2.000, 2.000, 2.000, 1.999, 1.964, 1.000, 0.036, 0.001, 0.000, 0.000, 0.000]';
mean_anal = mean(analytical_data_p);

for k = 1:length(mesh_resolutions)
    N = mesh_resolutions(k);
    fprintf('Running simulation with N = %d cells...\n', N);

    diag = false;
    case_study = "HDS";

    [Geometry, Fluid, Numerical, BoundaryConditions] = setupSimulation(diag);
    Numerical.N = N;
    Numerical.M = round(Geometry.H / Geometry.L * Numerical.N);

    num_cells(k) = Numerical.N * Numerical.M;

    [nod_x, nod_y, center_x, center_y, sx_w, sx_e, sx_p, sy_s, sy_n, sy_p] = generateMesh(Geometry, Numerical);

    A_x = Geometry.L / Numerical.N;
    A_y = Geometry.H / Numerical.M;

    x = nod_x - Geometry.H;
    [surf_x, surf_y] = meshgrid(x, fliplr(nod_y));
    surf_x = surf_x';
    surf_y = fliplr(surf_y');
```

```
41
42      [v_x, v_y] = computeVelocityField(surf_x, surf_y, Fluid, diag);
43
44      phi_def = zeros(Numerical.N+2, Numerical.M+2, length(Fluid.gamma));
45
46      for l = 1:length(Fluid.gamma)
47          phi_in = ones(Numerical.N+2, Numerical.M+2) * BoundaryConditions
                .phi_inlet;
48          phi_prev = phi_in;
49
50          [a_n, a_s, a_e, a_w, a_p, a_p_base, b_p] = computeCoefficients(
                v_x, v_y, Numerical, Fluid, ...
51                                                              sx_w
                                                                ,
                                                                sx_e
                                                                ,
                                                                sx_p
                                                                ,
                                                                sy_s
                                                                ,
                                                                sy_n
                                                                ,
                                                                sy_p
                                                                ,
                                                                ...
52                                                              Fluid
                                                                .
                                                                gamma
                                                                (
                                                                l
                                                                )
                                                                ,
                                                                case_study
                                                                ,
                                                                A_x
                                                                ,
                                                                A_y
```

```
                                                                    )
                                                                    ;


        [a_n, a_s, a_e, a_w, a_p, b_p] = applyBoundaryConditions(a_n,
            a_s, a_e, a_w, a_p, a_p_base, b_p, ...
                                                                surf_x
                                                                    ,
                                                                    Numerical
                                                                    ,
                                                                    diag
                                                                    ,
                                                                    BoundaryConditi
                                                                    );

        [phi, ~, ~] = solveSystem(a_n, a_s, a_e, a_w, a_p, a_p_base, b_p
            , ...
                                    phi_in, phi_prev, Numerical);

        phi_def(:, :, l) = phi;
    end

    for l = 1:length(Fluid.gamma)
        phi_n = zeros(length(analytical_data_x), 1);
        int_phi = interp1(nod_x - Geometry.H, phi_def(:, 1, l),
            analytical_data_x, 'linear', 'extrap');
        phi_n = int_phi(:);

        mae_values(k, l) = mean(abs(phi_n - analytical_data_p(:, l)))/
            mean_anal(l)*100;
    end

    fprintf('  MAE (rho/Gamma=10): %.6e\n', mae_values(k, 1));
    fprintf('  MAE (rho/Gamma=1e3): %.6e\n', mae_values(k, 2));
    fprintf('  MAE (rho/Gamma=1e6): %.6e\n\n', mae_values(k, 3));
end
%% PLOT

figure('Name', 'Grid Independence Study', 'NumberTitle', 'off');
hold on;
plot(mesh_resolutions, mae_values(:, 1), '-o', 'LineWidth', 2, '
    MarkerSize', 8, 'DisplayName', '$\rho/\Gamma = 10$');
plot(mesh_resolutions, mae_values(:, 2), '-s', 'LineWidth', 2, '
    MarkerSize', 8, 'DisplayName', '$\rho/\Gamma = 10^3$');
plot(mesh_resolutions, mae_values(:, 3), '-^', 'LineWidth', 2, '
    MarkerSize', 8, 'DisplayName', '$\rho/\Gamma = 10^6$');
hold off;
```

```
83
84 xlabel('Total Grid Cells (N $\times$ M)', 'Interpreter', 'latex', '
       FontSize', 12);
85 ylabel('Mean Absolute Error (%)', 'Interpreter', 'latex', 'FontSize',
       12);
86 title('Grid Independence Study - HDS Scheme', 'FontSize', 14);
87 legend('Interpreter', 'latex', 'Location', 'best', 'FontSize', 11);
88 grid on;
89 ax = gca;
90 ax.GridAlpha = 0.3;
91
92 exportgraphics(gca, 'figures/gridIndependenceStudy.png', 'Resolution',
       300, 'ContentType', 'image', 'BackgroundColor', 'white');
93
94 fprintf('=== Grid Independence Study Completed ===\n');
95 fprintf('Results saved to figures/gridIndependenceStudy.png\n');
```